



Python Programming Foundations

Week 2 - Decision Logic and Repetition
Day 2: "Repetition with Control"

1



Repeat On Purpose

Loops let a program repeat work instead of writing the same code again and again.
Good repetition is controlled repetition.

Today, watch for:

- ▶ what repeats
- ▶ what changes
- ▶ what stops the loop

2



Repeat an Action Several Times

Do the action, then check the condition.
Repeat while the condition is True. Stop when it is False.

This is a loop.
The action is repeated while the condition is True.
When the condition becomes False, the loop stops.

Repeat On Purpose

3



Logic Chooses, Loops Repeat

Decision logic chooses what happens.
Loop logic repeats something on purpose.

Week 2 now has two control-flow tools:

- ▶ branches for choosing
- ▶ loops for repeating

4



Two Ways to Control Flow

BRANCHES CHOOSE

Pick one path based on a condition.

```

graph TD
    A[Check Condition] -- True --> B[Do This (Path A)]
    A -- False --> C[Do That (Path B)]
    B --> D[Continue After the Choice]
    C --> D
            
```

Branches pick one path. Then the program continues.

LOOPS REPEAT

Repeat an action while a condition is True.

```

graph TD
    A[Check Condition] -- True --> B[Do the Action (Repeat This)]
    B --> A
    A -- False --> C[Stop Loop Continue After Loop]
            
```

Loops repeat the action. They stop when the condition is False.

Logic Chooses, Loops Repeat

5



What Counts as Success Today

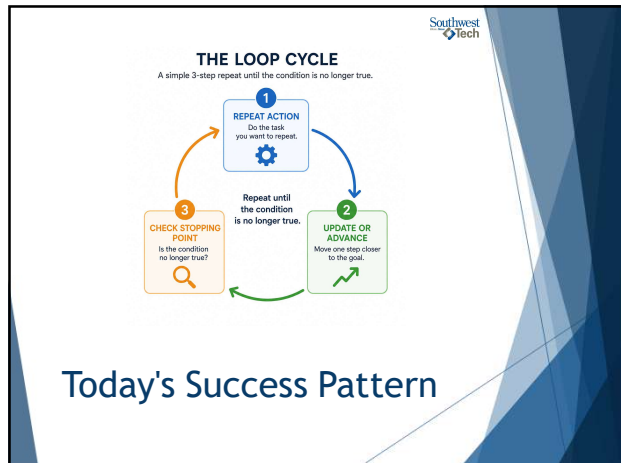
Repeat action -> update or advance
-> check stopping point.

If you can explain those three parts, you understand the loop.

For every loop, ask:

What code repeats?	What becomes different?	How does the loop stop?
--------------------	-------------------------	-------------------------

6



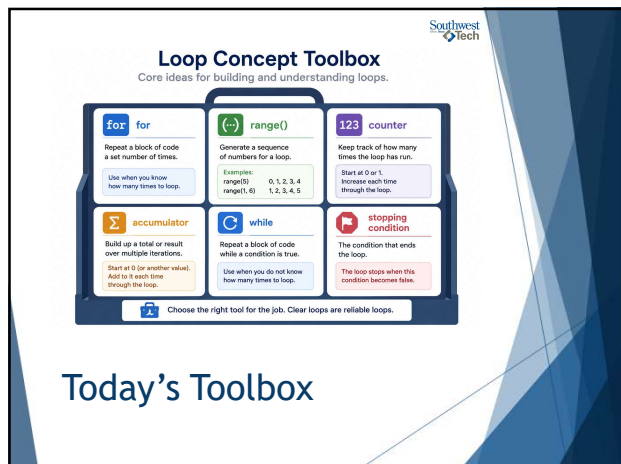
7

What We Will Use Today

- ▶ `'for'`
- ▶ `'range()'`
- ▶ counters
- ▶ accumulators
- ▶ `'while'`
- ▶ stopping conditions

These tools let a program repeat work without duplicating the same lines.

8



9

What We Will "Skip" For Now

- ▶ nested loops
- ▶ advanced iterators
- ▶ generator tools
- ▶ large menu programs

Instead: A good beginner loop is small enough to trace:

- ▶ first pass
- ▶ second pass
- ▶ last pass

10

Ask What Repeats

Loop Example

```

1 total = 0
2 for number in [1, 2, 3, 4, 5]:
3     print(number)
4     total = total + number
5 print("Total:", total)
  
```

The repeated block is the work the loop performs each time. Before running a loop, identify the repeated action.

Examples:

- ▶ print a number
- ▶ add to a total
- ▶ ask for another value
- ▶ count down toward zero

Annotations for the code example:

- Loop starts here (at line 2)
- Loop body (the repeated part) (lines 3-4)
- This runs after the loop ends (line 5)

11

Ask What Changes

Loop in Action: State Changes
The same steps run each pass, but the values change.

PASS 1
counter = 1
`print(counter)` → 1
counter = counter + 1
Output: 1

PASS 2
counter = 2
`print(counter)` → 2
counter = counter + 1
Output: 2

PASS 3
counter = 3
`print(counter)` → 3
counter = counter + 1
Output: 3

STATE CHANGES
The value of counter changes each pass (1 → 2 → 3 → ...). That is why the loop eventually stops.

A controlled loop usually has something that changes or advances.

That change may be:

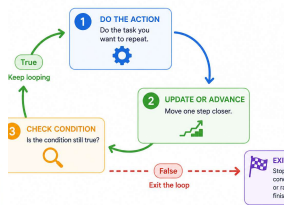
- ▶ the next number from `'range()'`
- ▶ a counter value
- ▶ a running total
- ▶ a user choice
- ▶ a condition becoming false

12

Ask What Stops It

Loop Cycle with Exit

Repeat the steps while the condition is true.



Every useful loop needs a clear stopping point.

The stop may come from:

- ▶ reaching the end of a range
- ▶ reaching the end of a sequence
- ▶ a condition becoming false
- ▶ a user choosing to exit

If nothing can stop, the loop may run forever.

`for` Loops: Known Repetition

A `for` loop is useful when the program knows what it is repeating over.

Common beginning pattern:

```
for number in range(5):
    print(number)
```

The range controls how many times the loop runs.

Demo #1

"For-Loop Counting"

What to watch for

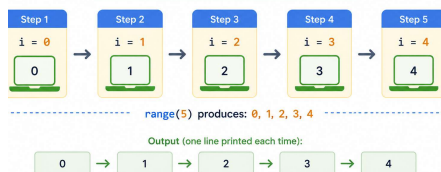
Watch the loop variable change each time the loop runs.

For each pass, identify:

- ▶ the current value
- ▶ the repeated output
- ▶ when the range is finished

range(5) in Action

```
for i in range(5):
    print(i)
```



Demo 1: Counting

Accumulators Build A Result

An accumulator is a variable that collects or builds a value over repeated steps.

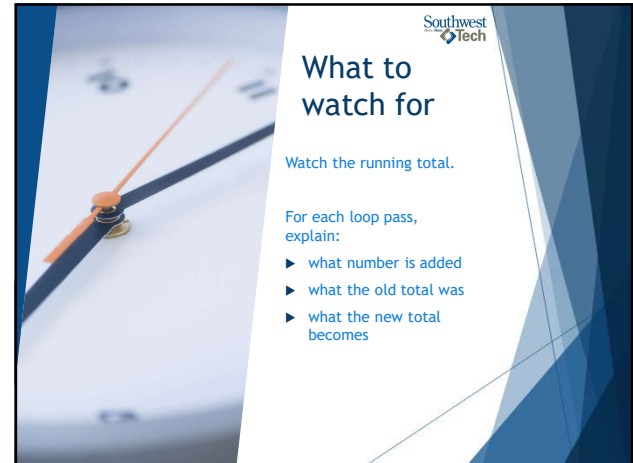
Common pattern:

```
total = 0
for number in range(1, 4):
    total = total + number
```

The total changes each time the loop runs.



19



20

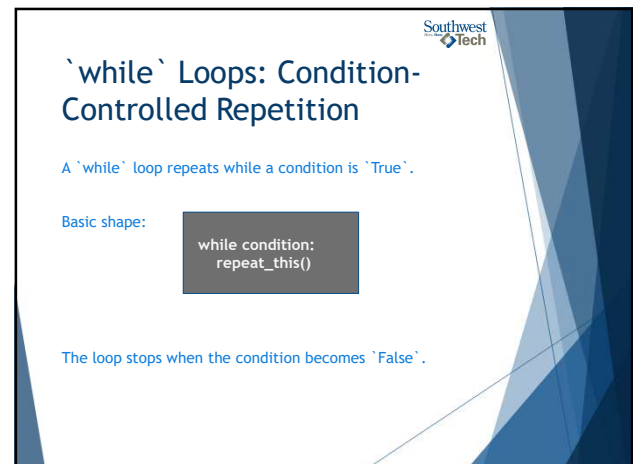
Running Total Across Loop Passes

pass	number	old total	new total (updated)
1	3	0	3
2	5	3	8
3	2	8	10

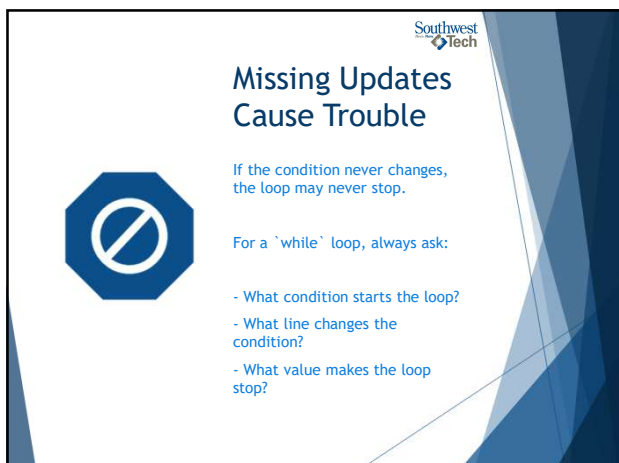
Each pass adds the number to the total. The new total is the updated (accumulated) result.

12 - Demo 2: Accumulator

21



22



23



24



What to watch for

Watch the countdown change.

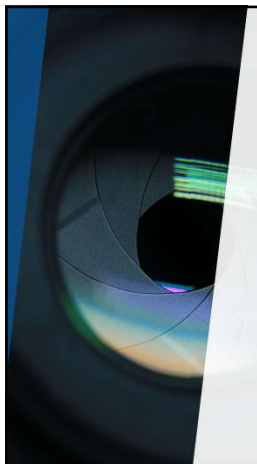
For each pass, identify:

- ▶ the current value
- ▶ the output
- ▶ the update
- ▶ the moment the loop stops

25



26



Optional Demo #4

"Input Loop w/Exit"

27



What to watch for

Repeated input is useful when the user may enter several values.

The program should still have a clear exit condition.

Watch for:

- ▶ the repeated prompt
- ▶ the user's choice
- ▶ the condition that ends the loop

28



From Demo to Lab

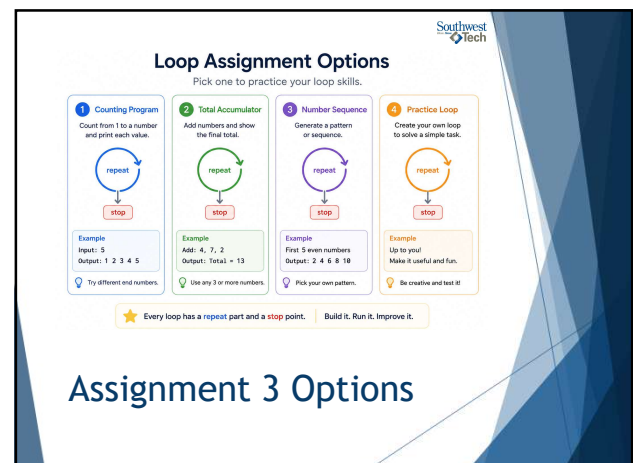
For Assignment 3, build one small program that repeats in a controlled way.

A good first version should include:

- ▶ one repeated action
- ▶ one clear stop point
- ▶ visible output
- ▶ no infinite loop

Before coding too much, write what repeats and what stops it.

29



30



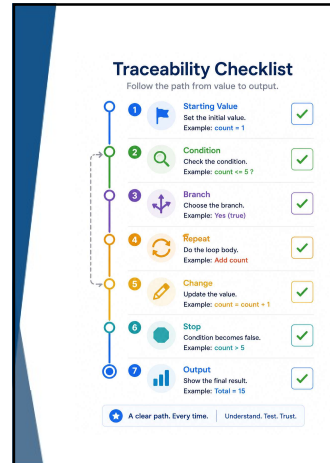
Closing Success Check

If you can explain the loop, you control it.

Before submitting Assignment 3, test:

- ▶ what happens on the first pass
- ▶ what changes after one pass
- ▶ how the loop stops
- ▶ whether the output proves repetition happened

31



Traceability Checklist

Follow the path from value to output.

1	Starting Value Set the initial value. Example: <code>count = 1</code>	✓
2	Condition Check the condition. Example: <code>count <= 5?</code>	✓
3	Branch Choose the branch. Example: Yes (true)	✓
4	Repeat Do the loop body. Example: <code>Add count</code>	✓
5	Change Update the value. Example: <code>count = count + 1</code>	✓
6	Stop Condition becomes false. Example: <code>count = 5</code>	✓
7	Output Show the final result. Example: <code>Total = 15</code>	✓

A clear path. Every time. Understand. Test. Trust.

Closing Success Check

(Example)

32



Thank you!

Have Fun!

33